

Thế tiến thoái lưỡng nan của điệp viên hai mang

- **Giới hạn thời gian:** 12 phút.
- **Dung lượng lưu trữ:** 5 GB
- **Môi trường:** một GPU (≈ 16 GB VRAM), không có internet
- **Dung lượng lời giải:** `solution.ipynb` ≤ 1 MB
- **Điểm baseline:** 0

Tại trung tâm AI quốc gia ở Astana, hai mô hình máy tính — Model R (một ResNet-18) và Model V (một ViT-Tiny) — đang phân tích ảnh. Hiện tại, cả hai mô hình đều hoạt động hoàn hảo, đạt độ chính xác 100% và đồng thuận trên từng ảnh. Để kiểm tra xem các “bộ não” thông minh của chúng thực sự khác nhau đến mức nào, nhà khoa học trưởng giao cho bạn một thử thách: thực hiện những thay đổi rất nhỏ, gần như không thể nhìn thấy đối với các pixel của mỗi ảnh sao cho Model R và Model V hoàn toàn bất đồng với nhau.



1. Nhiệm vụ

Hai bộ phân loại ảnh đã được huấn luyện trước xem cùng một ảnh. Trên các ảnh được cung cấp trong nhiệm vụ này, cả hai bộ phân loại đều đạt độ chính xác 100%.

- **Model R:** `torchvision.models.resnet18` (một CNN, ResNet18).
- **Model V:** `timm's vit_tiny_patch16_224` (một Transformer, ViT-Tiny).

Nhiệm vụ của bạn là tạo một thay đổi nhỏ (“nhiều”) cho mỗi ảnh sao cho hai mô hình bất đồng. Với mỗi ảnh, bạn phải tạo **hai nhiều khác nhau**:

- **Loại A:** sau khi cộng nhiễu, Model R vẫn phân loại ảnh chính xác, nhưng Model V phân loại ảnh sai.
- **Loại B:** sau khi cộng nhiễu, Model V vẫn phân loại ảnh chính xác, nhưng Model R phân loại ảnh sai.

Mỗi nhiễu phải đủ *nhỏ* để khó nhận thấy. Nhiễu càng nhỏ thì điểm càng cao (xem Mục 5). Nhiễu được áp dụng trực tiếp lên ảnh gốc ở cấp độ pixel.

2. Dữ liệu công khai

Một tập hợp ảnh được cung cấp cùng nhiệm vụ, được tổ chức thành hai phân hoạch — `train` (100 ảnh) và `test_public` (100 ảnh) — mỗi phân hoạch gồm các ảnh có độ phân giải khác nhau. Tất cả các ảnh đều thuộc 1000 lớp của ImageNet-1K và cả Model R lẫn Model V đều đạt độ chính xác 100% trên cả hai phân hoạch.

Các tệp sau được cung cấp:

```
train/images/*.png      # 100 ảnh ở định dạng PNG
train/labels.json       # ánh xạ mỗi chỉ mục ảnh tới nhãn đúng của nó
test_public/images/*.png # 100 ảnh ở định dạng PNG
test_public/labels.json  # ánh xạ mỗi chỉ mục ảnh tới nhãn đúng của nó
```

Trong quá trình chấm, thư mục `dataset/test_public/` của bạn được thay thế một cách trong suốt bằng hai tập ảnh ẩn (`test_leaderboard_a` và `test_leaderboard_b`) để chấm điểm chính thức. Mỗi tập chứa **100 ảnh** ở định dạng PNG và một tệp nhãn.

Lưu ý: Đối với nhiệm vụ này, có thể truy cập các nhãn trong các dataset kiểm thử.

3. Định dạng đầu ra

Với mỗi ảnh, bạn phải tạo hai tệp:

```
{index}_a.pt  # nhiễu loại A
{index}_b.pt  # nhiễu loại B
```

- `{index}` (0, 1, 2, ...), khớp với tên của ảnh trong các dataset.
- Mỗi tệp là một tensor duy nhất được lưu bằng `torch.save`. Kích thước của tensor phải là $3 \times H \times W$, trong đó H và W khớp với độ phân giải **gốc** của ảnh đó (không phải 224×224).
- Mã chỉ nên tạo ra một tệp ZIP, `submission.zip`. Đặt tất cả các tệp `.pt` ở cấp cao nhất của tệp lưu trữ ZIP, không có thư mục bao ngoài hoặc thư mục con.

```
submission.zip
├─ 0_a.pt
├─ 0_b.pt
├─ 1_a.pt
└─ ...
```

Notebook sẽ cảnh báo bạn nếu có bất kỳ vấn đề nào với định dạng đầu ra.

4. Các ràng buộc

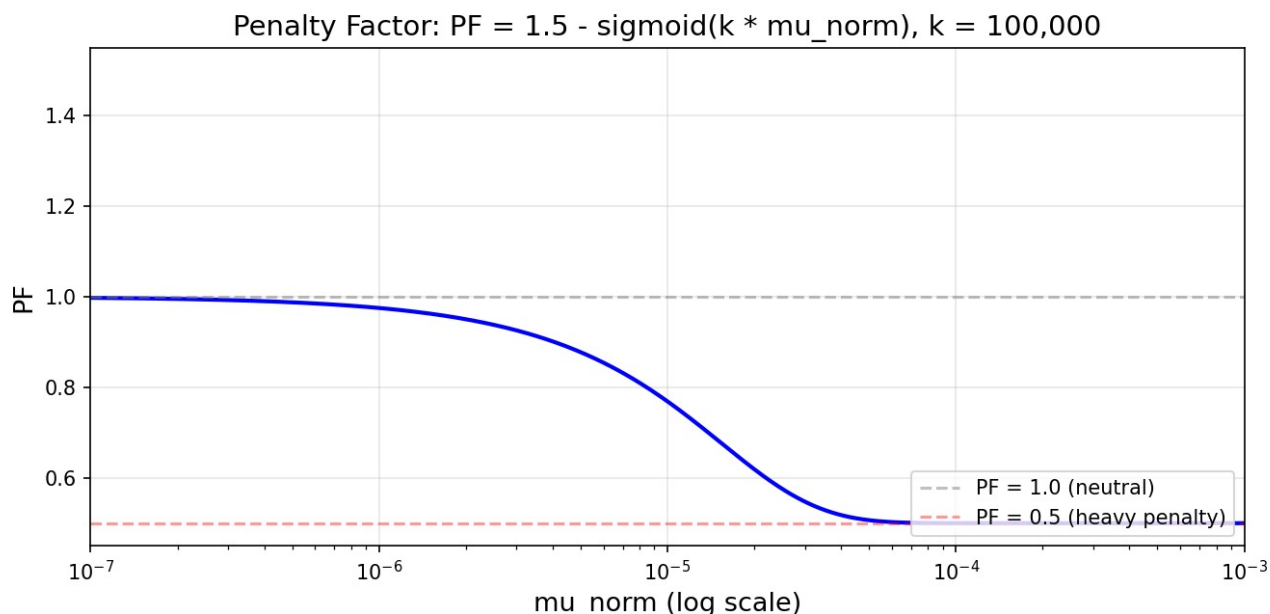
- **Mô hình:** Bạn phải sử dụng `torchvision.models.resnet18(pretrained=True)` và `timm.create_model('vit_tiny_patch16_224', pretrained=True)`. Không được phép sử dụng bất kỳ mô hình đã được huấn luyện trước nào khác.
- **Pipeline biến đổi (được bắt buộc khi đánh giá):** `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`. See Section 3 of `baseline.ipynb` để biết chi tiết.
- **Độ phân giải của nhiễu:** Phải khớp với độ phân giải ảnh thô **gốc** (không phải 224×224). Tensor được cộng vào ảnh thô *trước* pipeline biến đổi.
- **Định dạng đầu ra:** Chỉ các tệp `.pt` — không có PNG/JPG. Các tensor được cộng vào ảnh thô (đã được chuẩn hóa về `[0, 1]`) và các giá trị sau đó được cắt về `[0, 1]` trước khi tiền xử lý.
- **Đặt tên tệp:** Liệt kê phẳng, theo đúng định dạng `{index}_a.pt / {index}_b.pt`. Không có thư mục con bên trong tệp zip.
- **Thư viện:** `torch`, `torchvision`, `timm`.

5. Chấm điểm

Điểm cuối cùng được tính như sau. Gọi M là số lượng ảnh trong phân hoạch, $Score_A$ là số lượng nhiễu Loại A thành công, và $Score_B$ là số lượng nhiễu Loại B thành công:

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF là một hàm được thiết kế để phạt các nhiễu có norm cao và rất nhạy khi gần mức trần hiệu năng. Nó bị chặn trong khoảng từ 0.5 đến 1. Có thể xem phần triển khai đầy đủ trong Mục 8 của `solution.ipynb`.



Hình: Đường cong của hàm phạt.

6. Kiểm tra bài nộp

Notebook có các bước kiểm tra để cảnh báo bạn nếu có vấn đề về định dạng, tại Mục 7 trong notebook `solution.ipynb`.

7. Kiểm thử cục bộ

`solution.ipynb` chứa một ví dụ hoàn chỉnh và hoạt động được. Ví dụ này tải dữ liệu công khai, cả hai mô hình và trình chấm chính thức, rồi ghi một tệp ZIP bài nộp. Hãy đọc ví dụ này trước khi bắt đầu.

8. Cách nộp bài

- Lưu các thay đổi của bạn vào `solution.ipynb`.
- Mở tab Git trong thanh bên trái của JupyterLab.
- **Stage** `solution.ipynb` (biểu tượng + bên cạnh tệp).
- Nhập thông điệp commit và nhấp vào **Commit**.
- Nhấp vào biểu tượng đám mây có mũi tên hướng lên để push.
- Quay lại trang Cuộc thi này và nhấp vào **Submit**.

Nộp đúng một tệp có tên `solution.ipynb`.